

Como a tecnologia funciona

A matemática da similaridade, as estruturas de dados da seleção parcial e as decisões de concorrência por trás do recomendador, e do monólito modular que o hospeda.

89.740

FAIXAS NO ÍNDICE

9

DIMENSÕES

$O(n \cdot \log K)$

POR CONSULTA

43

REGRAS DE FRONTEIRA

.NET 8 · monólito modular (DDD + CQRS) · PostgreSQL, EF Core no write-side e Dapper no read-side
ML.NET · React 19 + Vite + ECharts

Companheiro de [ARQUITETURA.md](#), que cobre o fluxo e o mapa. Aqui o assunto é o **como**.

Sumário

01 O problema, em termos numéricos

Por que "parecido" precisa de definição formal

02 Normalização, o passo sem o qual tudo desanda

Escalas heterogêneas · z-score contra min-max · o piso de σ

03 Similaridade do cosseno

Direção contra magnitude · norma zero · a decomposição que soma o score

04 Seleção parcial: top-K sem ordenar o catálogo

Heap mínimo invertido · $O(n \cdot \log K)$ · determinismo por desempate

05 Over-fetch adaptativo

Por que o dedup encurta a lista · janela que dobra · por que o prefixo é exato

06 Blend com sinal colaborativo

Jaccard · reescala min-max · a contradição aparente com o capítulo 2

07 Colapso de quase-duplicatas

Duas chaves independentes · por que nenhuma sozinha basta

08 A engenharia ao redor do cálculo

Mediator e pipeline · ordem de publicação sob concorrência · Outbox · CQRS físico

09 Como o projeto se defende de si mesmo

Fronteiras verificadas · a armadilha da vacuidade · gates numéricos

10 Apêndice: números medidos

Catálogo, índice, latência e pegada em container

O problema, em termos numéricos

 $\mathbb{R}^9$

danceability, energy,
valence, tempo,
acousticness,
instrumentalness,
liveness, speechiness,
loudness

O recomendador responde a uma pergunta que parece simples: *dada uma faixa, quais são as mais parecidas?* A dificuldade é que "parecido" não significa nada até você escolher um espaço, uma métrica e uma regra de desempate. Cada uma dessas escolhas tem consequência observável no resultado, e este documento percorre todas elas.

Cada faixa vira um ponto em $\mathbb{R}^9$. As nove coordenadas são as features contínuas de áudio. A partir daí, o problema é inteiramente geométrico.

$$\text{faixa} \mapsto \mathbf{v} = (x_1, x_2, \dots, x_9) \in \mathbb{R}^9$$

O recomendador inteiro é uma sequência de decisões sobre como comparar dois pontos deste espaço.

Cada capítulo adiante responde três perguntas na mesma ordem: **o que faz, por que assim e o que quebra se fosse diferente**. A terceira é a que vale mais, porque quase todas as respostas vieram de um defeito real.

Normalização, o passo sem o qual tudo desanda

Amplitude

Tempo tem amplitude 243 vezes maior que energy. Nada nisso é musical: é só a unidade de medida.

2.1 O problema das escalas heterogêneas

As nove features não vivem na mesma escala, e a diferença é brutal:

FEATURE	FAIXA DE VALORES	AMPLITUDE
tempo	0 a 243 BPM	243
loudness	-50 a +5 dB	55
energy, valence, danceability, ...	0 a 1	1

Qualquer métrica que envolva produto de coordenadas fica dominada pelas duas de maior amplitude. Uma diferença de 40 BPM pesa quarenta vezes mais que a diferença máxima possível em *energy*.

O QUE QUEBRA SEM NORMALIZAR

O ranking vira, na prática, "faixas com BPM parecido", e as outras sete features viram ruído decorativo. O projeto tem um teste de dominância que prova os dois lados disso.

2.2 z-score, e não min-max

A transformação escolhida é a padronização z-score, aplicada coordenada a coordenada:

$$z_i = (x_i - \mu_i) / \sigma_i$$

μ e σ são a média e o desvio-padrão populacionais daquela feature, aprendidos numa passada única sobre o catálogo elegível.

Depois da transformação, cada coordenada tem média 0 e desvio 1. Nenhuma feature carrega mais peso por causa da unidade em que foi medida.

POR QUE NÃO MIN-MAX

Min-max ancora nos extremos, $(x - \min) / (\max - \min)$. Tempo e loudness têm cauda longa, então um único outlier comprime todo o resto do intervalo. O z-score se apoia em μ e σ , muito mais estáveis sob cauda. Decisão registrada como DP-1 do épico E4.1, com o teste de dominância como evidência.

$\sigma \approx 0$

Feature praticamente constante. Dividir por ela estouraria a coordenada ou geraria NaN.

2.3 O piso de σ , e por que ele é 1

```
public const double MinimumStandardDeviation = 1e-9;

standardDeviations[feature] = standardDeviation < MinimumStandardDeviation
    ? 1.0      // divide por 1: a coordenada centrada colapsa para ~0
    : standardDeviation;
```

POR QUE 1.0 E NÃO UM ÉPSILON

Dividir por um *épsilon* minúsculo transformaria ruído numérico no sinal *dominante* do espaço. Dividir por 1 mantém a coordenada centrada perto de zero, o que é honesto: uma feature sem variância não distingue faixa nenhuma, então não deve influenciar a comparação.

2.4 A invariante que sustenta tudo

Os parâmetros μ e σ são aprendidos **uma vez** e ficam fixados junto do índice. Semente e candidatos passam pela **mesma** transformação.

A FALHA SILENCIOSA QUE ISSO PREVINE

Normalizar a semente com um conjunto de parâmetros e os candidatos com outro produz números plausíveis e sem sentido. É a mesma classe de erro que o skew treino/inferência causa em modelos supervisionados: nada estoura, tudo fica sutilmente errado.

Similaridade do cosseno

3.1 Direção, não magnitude

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\sum_i a_i b_i}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

Resultado em $[-1, 1]$. Vale 1 quando os vetores apontam exatamente na mesma direção.

POR QUE COSSENO E NÃO DISTÂNCIA EUCLIDIANA

O cosseno mede semelhança de **perfil**, ignorando magnitude. Duas faixas com o mesmo caráter sonoro, uma sistematicamente mais intensa que a outra, apontam na mesma direção e têm cosseno alto. A distância euclidiana as separaria pela intensidade, que não é o que se quer num recomendador content-based.

3.2 O vetor de norma zero

O cosseno é indefinido quando um dos vetores é a origem: não há direção para comparar. O código devolve 0, não NaN. Devolver 0 significa "nenhuma semelhança de direção", leitura defensável que mantém a aritmética fechada. No espaço z-score isso só ocorre com uma faixa exatamente na média das nove features.

3.3 A explicabilidade que não pode divergir do ranking

Esta é, na minha leitura, a decisão mais elegante do projeto. A explicação de *por que* duas faixas se parecem não é calculada à parte: ela é a **decomposição** do próprio score.

$$c_i = (a_i \cdot b_i) / (\|\mathbf{a}\| \cdot \|\mathbf{b}\|) \Rightarrow \sum_i c_i = \cos(\mathbf{a}, \mathbf{b})$$

Cada parcela é a contribuição da i -ésima feature, e a soma reconstrói exatamente o score.

A prova é imediata: o denominador é constante em relação a i , então somar as parcelas é somar o numerador do produto escalar e dividir pelo mesmo denominador.

NaN

Envenena toda comparação que toca, e o efeito aparece longe da causa.

A PROPRIEDADE QUE ISSO GARANTE

"As features que mais aproximaram" são, por construção, as de maior parcela. Como a soma reconstrói o score, é **impossível** a explicação apontar para um lado e o ranking para outro. A interface exibe as três maiores parcelas por item.

3.4 O score híbrido

O ranking final do caminho content-based não é o cosseno puro: soma um bônus quando candidato e semente compartilham gênero.

$$\text{score} = \cos(\mathbf{a}, \mathbf{b}) + \text{bônus}, \quad \text{bônus} = 0,05 \text{ se mesmo gênero; } 0 \text{ caso contrário}$$

MODO	ELEGIBILIDADE	BÔNUS
<code>boost</code> (padrão)	todos elegíveis	+0,05 para mesmo gênero
<code>off</code>	todos elegíveis	nenhum, cosseno puro
<code>sameGenreOnly</code>	filtro duro: só mesmo gênero	não se aplica

GÊNERO IMPUTADO NÃO CONTA

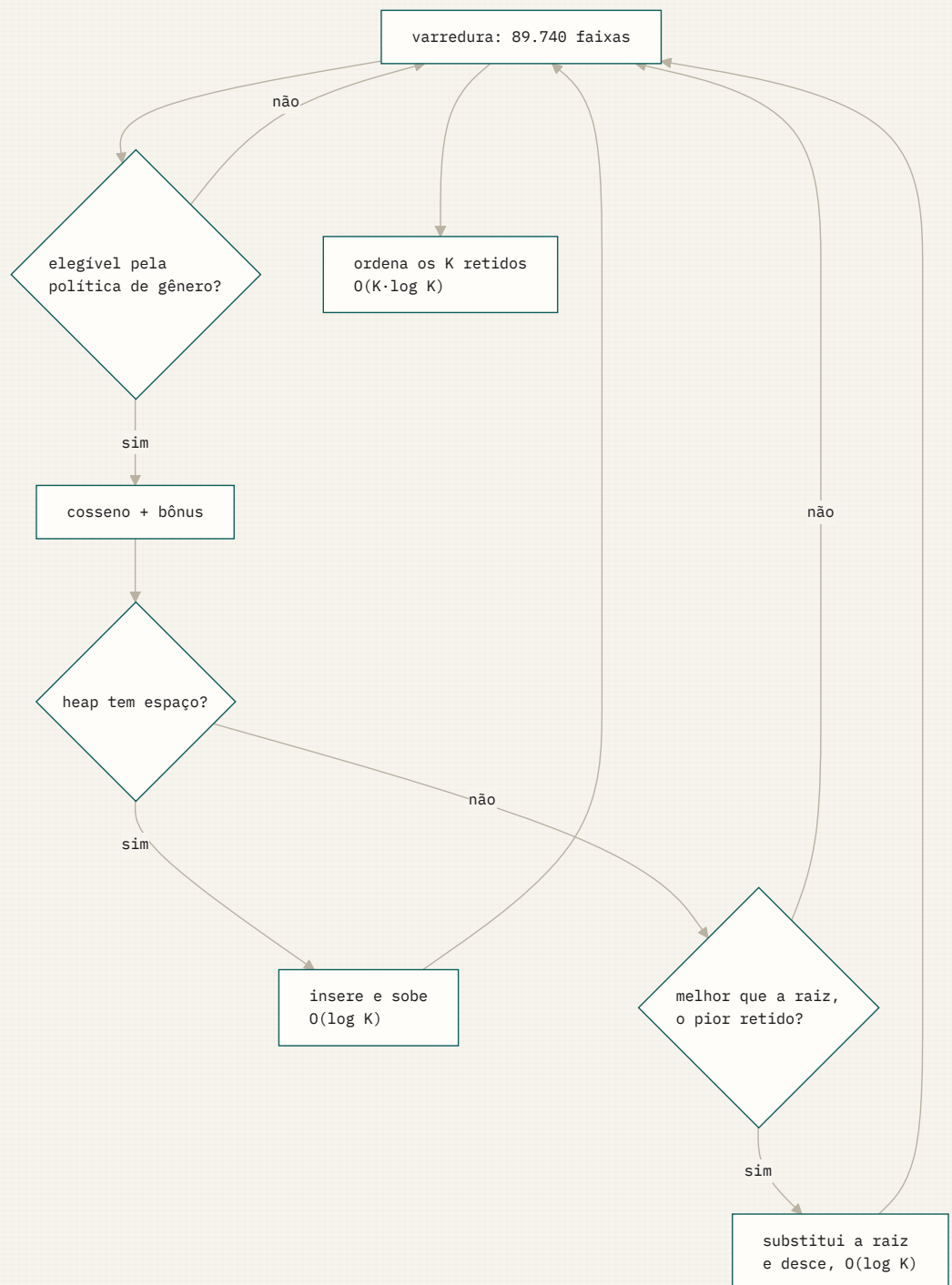
Se o gênero da semente for nulo, vazio ou *imputado*, a política degrada para `off` e sinaliza `genreFellBackToCosineOnly` na resposta. Candidata com gênero imputado também não recebe bônus, mesmo que o rótulo coincida. O critério é explícito: **estimado não é medido**, e o usuário é informado quando a degradação acontece.

Seleção parcial: top-K sem ordenar o catálogo

0 ingênuo

Ordenar 89.740 elementos para devolver 10 e descartar 99,99% do trabalho.

Calcular o cosseno contra as 89.740 faixas e ordenar tudo custa $O(n \cdot \log n)$ e aloca a lista inteira. O código usa seleção parcial com um heap **mínimo** de capacidade K . A inversão é o truque: manter na raiz o *pior* dos K retidos permite decidir em $O(1)$ se um candidato novo merece entrar.



$O(n \cdot \log K)$ em vez de $O(n \cdot \log n)$

Com $n = 89.740$ e $K = 120$: $\log_2 K \approx 7$ contra $\log_2 n \approx 16,5$. Mais importante, a memória retida é $O(K)$, não $O(n)$.

4.1 O desempate, que não é detalhe

```
private static bool IsBetterThan(TrackSimilarity candidate, TrackSimilarity refere
{
    if (candidate.Similarity > reference.Similarity) return true;
    if (candidate.Similarity < reference.Similarity) return false;
    return string.CompareOrdinal(candidate.TrackId, reference.TrackId) < 0;
}
```

NÃO É PREFERÊNCIA DE NEGÓCIO, É REPETIBILIDADE

Sem o desempate, a ordem de dois empatados dependeria da ordem em que a varredura os encontrou, que depende da ordem de leitura do banco. O ranking deixaria de ser reprodutível e nenhuma medição de qualidade seria comparável entre execuções.

A CONSEQUÊNCIA QUE DESTRAVA O CAPÍTULO 5

Como o desempate é **total**, não existem dois itens indistinguíveis, e a lista ordenada é única. Logo, *o prefixo de tamanho w é exatamente o top- w* . É esta propriedade que permite o over-fetch adaptativo sem revarrer o catálogo.

Over-fetch adaptativo

Sintoma

Usuário pede 10, recebe 4. As outras 6 eram versões da mesma faixa.

O usuário pede 10 recomendações. O sistema busca 10, aplica o colapso de quase-duplicatas e entrega quatro: as outras seis eram remaster, ao vivo e edição deluxe da mesma música, e colapsaram num representante só.

sementes que recebiam menos de 10, de 1.895	679 (35,8%)
sementes que recebiam exatamente 1	227
tamanho médio da lista entregue	7,87

5.1 A solução, e a restrição que a molda

A saída óbvia é buscar mais candidatas que o limite. A restrição é que a varredura é $O(n)$ sobre 89.740 faixas: repetir por rodada triplicaria o custo num host de free tier. O desenho resolve as duas coisas ao varrer **uma vez na janela máxima** e pós-processar um **prefixo** por rodada.

$$\text{janela}(r) = \min(\max(\text{limit} \cdot 3, 10) \cdot 2^{(r-1)}, 600), \quad r \in \{1, 2, 3\}$$

Para limit = 10: janelas de 30, 60 e 120. A varredura acontece uma vez, já em 120.

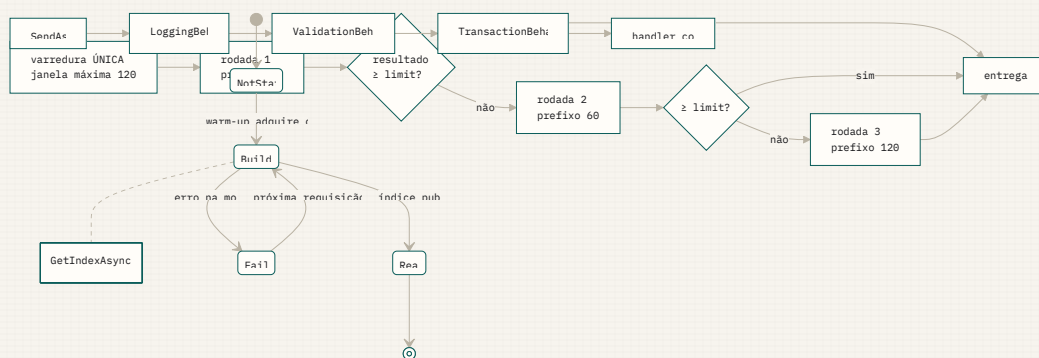


FIG. 2 — RODADAS SOBRE PREFIXOS DE UMA VARREDURA ÚNICA

POR QUE O PREFIXO É EXATO, E NÃO APROXIMADO

Pela propriedade do capítulo 4: o desempate é total, a lista ordenada é única, e o prefixo de tamanho w é o top- w que uma segunda varredura devolveria. O reaproveitamento não troca precisão por velocidade; é matematicamente equivalente.

5.2 A janela corta as duas fontes

O BUG QUE ISSO CORRIGIU

Enquanto só a lista de áudio era recortada, a rodada 1 disputava com o top-120 colaborativo em vez do top-30. Como a pior candidata de áudio da janela recebe score normalizado 0 (capítulo 6), qualquer colaborativa extra com Jaccard positivo a deslocava do top-10. O resultado mudava para sementes que já estavam saudáveis, sem nenhuma rodada extra ter ocorrido. Corrigido no commit `b226f7a`.

5.3 O resultado medido

SOBRE AS MESMAS 1.895 SEMENTES	ANTES	DEPOIS
com menos de 10 recomendações	679	0
com exatamente 1 recomendação	227	0
tamanho médio da lista	7,87	10,00

Distribuição de esforço: **64,2%** resolvem na rodada 1, **29,7%** na rodada 2, **6,2%** na rodada 3, e nenhuma bate no teto sem resolver. O custo é p50 +1,8 ms e média +2,8 ms; a variação no p95 fica dentro do ruído.

Blend com sinal colaborativo

1.481.511

pares na matriz, 254 MB, cobrindo 10,8% do catálogo

Além da similaridade de áudio existe o sinal colaborativo: duas faixas que aparecem juntas em muitas playlists provavelmente combinam, mesmo que sonoramente sejam distintas. A medida é o índice de Jaccard.

$$J(A, B) = \frac{|P_A \cap P_B|}{|P_A \cup P_B|}$$

P_X é o conjunto de playlists que contêm a faixa X. Resultado em $[0, 1]$.

A matriz é pré-computada e guardada como par canônico `track_id_low` / `track_id_high`, o que corta o armazenamento pela metade ao custo de a consulta precisar procurar a semente nos dois lados.

6.1 Combinar duas escalas incomparáveis

O score de conteúdo vive aproximadamente em $[-1, 1]$ mais o bônus; o Jaccard vive em $[0, 1]$, com distribuição completamente diferente. Somar direto faria uma parcela dominar por magnitude, não por mérito.

$$\text{contentNorm} = (\text{score} - \text{min}) / (\text{max} - \text{min})$$

$$\text{final} = (1 - w) \cdot \text{contentNorm} + w \cdot J$$

w é o `blendWeight`, padrão 0,35. A reescala acontece dentro do conjunto de candidatas daquela consulta.

POR QUE MIN-MAX AQUI, SE O CAPÍTULO 2 O REJEITOU

São problemas diferentes. Lá, min-max seria aplicado ao espaço global de features, onde outliers de cauda longa o tornam instável. Aqui ele age sobre um conjunto pequeno e já filtrado, e o objetivo não é preservar distribuição: é trazer duas escalas para a mesma faixa antes de combiná-las. A transformação é afim e **monótona**, então a ordem relativa dentro da parcela de conteúdo é preservada, inclusive o efeito do bônus de gênero.

Se todas as candidatas tiverem o mesmo score, `max - min = 0` e a reescala dividiria por zero. O código trata explicitamente, devolvendo 0 ou 1, em vez de propagar `NaN`.

6.2 O score exposto é o score que ordenou

O BUG QUE DEFINIU ESTA REGRA

O blend reordenava pelo cosseno *reescalado* em vez do score híbrido: o bônus de gênero era calculado e depois descartado na ordenação. O efeito medido foi de $-5,15\sigma$ na coerência de gênero, e passou dois épicos despercebido porque o próprio instrumento de medição continha o mesmo erro. Hoje o contrato é explícito: existe **um número só**, que ordena e que aparece na resposta.

ESTRATÉGIA	SCORE	SIGNIFICA	A SOMA COSINE+GENRE FECHA?
content	cosseno + bônus de gênero		sim
blend	score final do blender, normalizado		não

Colapso de quase-duplicatas

O catálogo tem a mesma música várias vezes: remaster, ao vivo, edição deluxe, single e álbum. Entregar as cinco como recomendações distintas é inútil. O colapso usa **duas chaves independentes**, e nenhuma resolve sozinha.

7.1 Chave numérica: cosseno quase perfeito

mesma faixa se $\cos(a, b) \geq 0,999$

Limiar deliberadamente severo: significa "features praticamente idênticas", não "parecidas".

7.2 Chave textual: nome normalizado

Normalização

corta sufixo após " -", remove parênteses e colchetes, tira acentos e pontuação, colapsa espaços

"Creep - Radio Edit"	→ "radiohead creep"
"Creep (Acoustic Version)"	→ "radiohead creep"
"Creep [2008 Remaster]"	→ "radiohead creep"

POR QUE AS DUAS CHAVES

O cosseno sozinho perde o remaster cujas features foram medidas de novo e diferem na terceira casa. A chave textual sozinha perde a mesma gravação publicada com títulos diferentes, e agrupa faixas homônimas de artistas distintos se a normalização errar. Juntas, cobrem os dois eixos de falha.

7.3 A ordem importa

O colapsador abre grupos **enquanto** o número de grupos for menor que o limite, percorrendo as candidatas na ordem do ranking. Não é um pós-filtro. Isso preserva a posição: o representante fica no lugar da melhor versão do grupo, e o contador `equivalentVersionsCollapsed` informa quantas foram absorvidas.

8.1 CQRS com mediator próprio

Sem MediatR

Resolução por reflexão
com cache em
`ConcurrentDictionary<Type,
MethodInfo>`

FIG. 3 – ANINHAMENTO REAL DOS BEHAVIORS

A ORDEM É CONTRAINTUITIVA

O primeiro behavior registrado vira o mais externo. Como `LoggingBehavior` é registrado antes, log envolve validação, e uma requisição que falha na validação *aparece* no log. O `CLAUDE.md` documenta a ordem invertida; o código é a fonte da verdade.

8.2 Servir um índice que ainda não existe

O índice é singleton, custa uma varredura completa para montar e sobe em background no arranque. Isso cria uma janela em que o processo aceita HTTP mas não consegue responder. O desenho torna esse estado **explícito** em vez de inferido.

FIG. 4 – ESTADO EXPLÍCITO DE DISPONIBILIDADE DO ÍNDICE

Três detalhes de concorrência que valem estudo:

1. **A ordem de publicação.** `_index` é escrito *antes* de `_state = Ready`. Com ambos `volatile`, a escrita volátil tem semântica de *release*: quem lê `Ready` tem garantia de enxergar o índice. A ordem inversa criaria uma janela com estado pronto e referência nula.

2. O estado não é inferido do semáforo. A versão anterior deduzia "indisponível" de `_gate.CurrentCount == 0`, detalhe de implementação do `SemaphoreSlim`. Isso produzia um 503 lançado depois de o índice já estar pronto, e uma requisição chegando antes do warm-up que *pagava a varredura inteira de forma síncrona*, com o warm-up enfileirado atrás dela.
3. A sinalização acontece no construtor. O hosted service avisa o provider no construtor, não no `ExecuteAsync`. O host resolve todos os `IHostedService` antes de subir o Kestrel, então quando a primeira requisição chega o provider já sabe que um warm-up vem a caminho.

8.3 CQRS levado à separação física

	WRITE - SIDE	READ - SIDE
Tipo	<code>ICommand</code>	<code>IQuery</code>
Acesso	EF Core, com rastreamento	Dapper, SQL à mão
Organização	agregado no Domain	Query, Handler e Result no mesmo arquivo
Transação	<code>TransactionBehavior</code>	nenhuma
Paginação	não se aplica	<code>ROW_NUMBER() OVER()</code> + <code>COUNT(*) OVER()</code>

A ARMADILHA DO COUNT

`COUNT(*) OVER()` do PostgreSQL devolve `bigint`, que materializa como `long`. Tipar como `int` compila, passa nos testes de contrato (que afirmam apenas o texto do SQL) e produz HTTP 500 em 100% das chamadas. Custou um endpoint inteiro quebrado com 18 testes verdes.

8.4 Outbox: entregar sem perder e sem travar

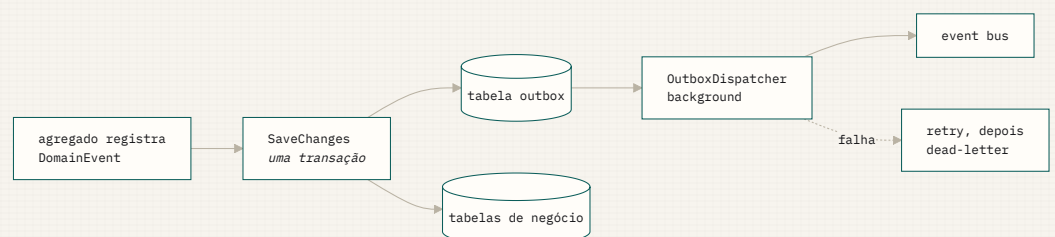


FIG. 5 — ENTREGA TRANSACIONAL POR OUTBOX

A GARANTIA QUE ISSO COMPRA

Ou a mudança de estado e o registro do evento acontecem juntos, ou nenhum dos dois: não existe o caso "salvou mas não avisou". E falha na publicação não derruba a operação principal, vira nova tentativa. A contrapartida é entrega *ao menos uma vez*, então o consumidor precisa ser idempotente.



Como o projeto se defende de si mesmo

São 43 regras ArchUnitNET que falham o `dotnet test` no instante em que uma fronteira é violada: Domain de um módulo tocando outro, Domain referenciando EF Core ou ASP.NET, SharedKernel referenciando infraestrutura, Host referenciando Domain interno.

9.1 A armadilha da vacuidade

Verde vazio

Uma regra sobre conjunto vazio passa sempre. Nada no resultado distingue "verificou e aprovou" de "não verificou nada".

QUATRO REGRAS QUE NÃO TESTAVAM NADA

Durante vários épicos, quatro das regras avaliavam zero tipos.

`ResideInAssembly(string)` compara com o `FullName` do assembly e recebia o nome simples: filtro vazio. Com `WithoutRequiringPositiveResults()` por cima, o verde era silencioso. O que segurava a pureza do SharedKernel era o `.csproj` sem referências, não o teste.

Existe hoje uma regra cuja única função é impedir a reincidência: ela verifica que os assemblies foram efetivamente descobertos. E ficou o protocolo: regra nova só conta como provada depois de ser vista vermelha com uma violação injetada de propósito.

9.2 Gates numéricos sobre o recomendador

LIMIAR	RÉGUA	MEDIDO
coerência de gênero	$\geq 0,40$	0,5563
ganho sobre cosseno puro	$\geq 2,0\times$	4,88×
saturação de gênero	$\leq 0,50$	0,2800
recall de duplicatas	$\geq 0,45$	0,6464
alcance de duplicatas	$\geq 0,50$	0,7037
cosseno médio entre irmãs	$\geq 0,98$	0,999028
repetição no top-N pós-dedup	$= 0$	0 de 1.895
taxa de lista curta	$\leq 0,05$	0,0000
sementes com um item só	$= 0$	0

UM LIMIAR SÓ VALE NA POPULAÇÃO QUE O CALIBROU

A taxa de lista curta foi calibrada sobre os 111 grupos de 11 ou mais quase-duplicatas, onde o fenômeno é real (35,8%). Numa amostra sorteada do catálogo ele é praticamente zero, então o mesmo limiar de 5% seria satisfeito trivialmente. Por isso a medição carrega **identidade de população**, e o gate recusa medição que não venha da população de calibração. Sem isso, trocar a amostra deixaria o gate verde mesmo revertendo a correção que ele existe para proteger.

Números medidos

Catálogo e índice

faixas no catálogo	89.740
faixas com features imputadas	0
artistas distintos	29.811
gêneros distintos	113
dimensão do espaço de similaridade	9
tempo de montagem do índice	1,05 a 1,68 s
memória retida pelo índice	39,7 a 40,8 MB

Matriz de co-ocorrência

pares armazenados	1.481.511
ocupação em disco	254 de 388 MB do banco
cobertura do catálogo	9.664 faixas (10,8%)

Latência por recomendação, endpoint local em Release

PERCENTIL	CONTENT	OBSERVAÇÃO
p50	10,60 a 11,71 ms	a varredura é ~96% do tempo
p95	14,15 a 15,07 ms	
p99	16,15 a 16,50 ms	

Pegada em container

TIER	RAM APÓS 1.800 REQUISIÇÕES	OOM
512 MB, sem limite de CPU	310,9 MiB	não
256 MB, sem limite de CPU	148,3 MiB	não

O QUE ESTES NÚMEROS NÃO DIZEM

A medição em container rodou **sem limite de CPU**, e o gargalo real do free tier é CPU, não RAM. Latência e cold start em container ainda não foram medidos com metodologia confiável, e por isso não aparecem aqui.

SpotifyDataAnalysis · documento técnico · gerado de
documentacao/arquitetura-tecnica.html por build-pdf.ps1
Companheiro de ARQUITETURA.md, que cobre fluxo e mapa, e de README.md, que cobre resultados.
Todos os números vêm de execução real contra o catálogo de 89.740 faixas.
Tipografia: Fraunces, Spectral e IBM Plex Mono.